

Manual Técnico

Centro de Rescate Animal: Gestión de Refugio y Adopciones

Autor: Greys Hernández **Carné:** 202503071 **Curso:** Introducción a la Programación y Computación 1 **Lenguaje:** Java 17+ **Interfaz:** Java Swing (construida por código, sin GUI builders)

1. Arquitectura del sistema

El proyecto sigue una arquitectura en capas (estilo MVC), separada en cuatro paquetes dentro de `src/`:

`src/`

- |— `Main.java` → punto de entrada
- |— `model/` → clases de datos y almacén estático
- |— `controller/` → lógica de negocio, validaciones, persistencia, reportes
- |— `view/` → ventanas y paneles Swing
- |— `util/` → utilidades de validación y fechas

Flujo general: `Main` carga los datos persistidos → muestra `LoginVista` → al autenticar, abre `MainVista`, que contiene un `CardLayout` con un panel por módulo. Cada panel (`view`) solo dibuja componentes y delega toda la lógica a su controlador correspondiente (`controller`), que a su vez opera sobre los arreglos estáticos definidos en `Almacen` (`model`).

2. Clases principales y responsabilidad

Paquete	Clase	Responsabilidad
model	<code>Almacen</code>	Contiene todos los arreglos estáticos y la matriz de

Paquete	Clase	Responsabilidad
		ubicaciones. Es el único lugar donde vive el estado del sistema.
model	<code>Animal</code> , <code>Adoptante</code> , <code>Solicitud</code> , <code>Rescate</code> , <code>Usuario</code> , <code>Evento</code>	Representan un registro individual de cada entidad; incluyen <code>aLineaArchivo()</code> (persistencia) y <code>aFilaTabla()</code> (para las JTable).
controller	<code>AuthControlador</code>	Login, control de intentos fallidos y bloqueo de sesión.
controller	<code>AnimalControlador</code>	Alta, búsqueda, edición de estado y baja lógica de animales.
controller	<code>AdoptanteControlador</code>	Alta, búsqueda (por código y DPI), edición de adoptantes.
controller	<code>SolicitudControlador</code>	Alta de solicitudes, aprobación (con rechazo automático de otras pendientes del mismo animal), rechazo.
controller	<code>RescateControlador</code>	Alta de rescates, atención de casos (genera o vincula animal), listado priorizado.
controller	<code>UbicacionControlador</code>	Asignar/liberar celdas de la matriz, validar capacidad, liberar automáticamente por baja lógica.
controller	<code>ArchivoUtil</code>	Lectura/escritura de todos los <code>.txt</code> de persistencia.
controller	<code>ReporteHTML</code>	Generación de los 4 reportes HTML.

Paquete	Clase	Responsabilidad
controller	Bitacora	Registro de acciones y errores del sistema en memoria y disco.
util	Validador	Formatos de código, DPI, teléfono, nombre, rangos numéricos.
util	FechaUtil	Validación de fechas dd/mm/aaaa.

3. Arreglos estáticos y matrices utilizados

Todos definidos en `model.Almacen`, con tamaño máximo fijo (sin estructuras dinámicas):

Arreglo	Tipo	Tamaño máx.	Contador asociado
animales	Animal[]	100	cantidadAnimales
adoptantes	Adoptante[]	(definido en Adoptante.MAX)	cantidadAdoptantes
solicitudes	Solicitud[]	(definido en Solicitud.MAX)	cantidadSolicitudes
rescates	Rescate[]	(definido en Rescate.MAX)	cantidadRescates
usuarios	Usuario[]	(definido en Usuario.MAX)	cantidadUsuarios
acciones / errores	Evento[]	(definido en Evento.MAX)	cantidadAcciones / cantidadErrores

Matriz de ubicaciones

```
public static final int FILAS = 2;    // fila 0 = Zona Perros, fila 1 = Zona Gatos
```

```
public static final int COLUMNAS = 10; // 10 jaulas por zona
```

```
public static String[][] ubicaciones = new String[FILAS][COLUMNAS];
```

- Cada celda `ubicaciones[f][c]` guarda el **código del animal** ocupante, o `null` si está libre.
 - Capacidad máxima por zona = número de columnas de esa fila = **10**.
 - No se usa ninguna colección del Java Collections Framework en ningún módulo (verificado: cero coincidencias de `ArrayList`, `HashMap`, `List`, `Queue`, `Stack`, `Vector`).
-

4. Validaciones implementadas

Campo	Regla	Clase responsable
Códigos (A-, AD-, S-, R-)	Prefijo + guion + dígitos, vía regex <code>^prefijo-\d+\$</code>	<code>Validador.esCodigo</code>
Nombre (adoptante)	Solo letras (con tildes/ñ) y espacios	<code>Validador.esSoloLetras</code>
DPI	13 dígitos numéricos, único	<code>Validador.esDPI + AdoptanteControlador.b</code> <code>uscarPorDpi</code>
Teléfono	8 dígitos numéricos	<code>Validador.esTelefono</code>
Edad estimada	Entero entre 0 y 25	<code>Animal.EDAD_MINIMA /</code> <code>EDAD_MAXIMA</code>
Estado clínico	<code>EN_OBSERVACION,</code> <code>EN_TRATAMIENTO, APTO</code>	<code>Animal</code> (constantes)
Estado adopción	<code>DISPONIBLE, ADOPTADO,</code> <code>ELIMINADO</code>	<code>Animal</code> (constantes)
Fecha	<code>dd/mm/aaaa</code> , calendario válido	<code>FechaUtil.esFechaValida</code>
Duplicados	Código único por módulo; DPI único en adoptantes	Cada <code>*Controlador</code>
Reglas de negocio	1 sola solicitud APROBADA por animal; animal debe estar	<code>SolicitudControlador,</code> <code>UbicacionControlador</code>

Campo	Regla	Clase responsable
	DISPONIBLE para recibir solicitud; celda ocupada no se reasigna	

Todo intento fallido se registra automáticamente en `Bitacora.error(...)`.

5. Persistencia en archivos

Carpeta `Proyecto1/datos/`, separador `|`, un registro por línea:

Archivo	Estructura
<code>animales.txt</code>	<code>codigo nombre especie edad estado Clinico estadoAdopcion fechaIngreso</code>
<code>adoptantes.txt</code>	<code>codigo nombre dpi telefono</code>
<code>solicitudes.txt</code>	<code>codigo codigoAnimal codigoAdoptante fecha estado</code>
<code>rescates.txt</code>	<code>codigo prioridad estado fecha codigoAnimalVinculado</code>
<code>bitacora_acciones.txt</code>	<code>fecha usuario modulo evento descripcion</code>
<code>bitacora_errores.txt</code>	<code>fecha usuario modulo evento motivo</code>

La carga ocurre al iniciar (`Main.java`) y también bajo demanda desde el panel de Reportes ("Cargar todo desde archivos"). El guardado ocurre al cerrar la ventana principal (`WindowListener` en `MainVista`) y bajo demanda ("Guardar todo en archivos").

6. Reportes HTML (`controller.ReporteHTML`)

Método	Reporte
<code>generarReporteAnimales()</code>	Listado completo de animales activos
<code>generarReporteAdopciones()</code>	Animales con estado <code>ADOPTADO</code>
<code>generarReporteOcupacion()</code>	Estado de la matriz de ubicaciones por zona
<code>generarReporteBitacora(acciones, errores)</code>	Bitácora combinada o filtrada

Los archivos se guardan en `Proyecto1/reportes/` con nombre `reporte_<tipo>_<dd-MM-yyyy>_<HHmm>.html`.

7. Módulo de autenticación

- Usuarios cargados en memoria (no desde archivo), en `AuthControlador.cargarUsuariosDePrueba()`:
 - `admin1 / Refugio2026` → rol `ADMIN`
 - `auxiliar1 / Auxiliar26` → rol `AUXILIAR`
 - Máximo 3 intentos fallidos → bloqueo de la sesión hasta reiniciar la aplicación (el bloqueo vive en memoria, no se persiste).
-

8. Cómo compilar y ejecutar

```
cd Proyecto1
```

```
javac -d out $(find src -name "*.java")
```

```
java -cp out Main
```

Requiere Java 17 o superior. El programa crea automáticamente la carpeta `datos/` si no existe.

9. Limitaciones conocidas / pendientes

- El módulo de rescates no cuenta aún con datos de prueba persistidos (`rescates.txt` vacío al momento de este manual).
- No se generaron aún ejemplos de `reporte_adopciones` ni `reporte_ocupacion` en `reportes/` (la funcionalidad existe y está probada por código, falta correrla con datos reales antes de la entrega).